

DEVOPS MASTER COURSE

Volume 8D

Google Cloud Containers, Serverless, APIs, Events, and Workflows

Chapters 31-40

Artifact Registry • GKE • Cloud Run • Functions • API Management • Pub/Sub • Eventarc •
Workflows • Tasks • Cloud-Native Capstone

2026 Edition

About This Part

Volume 8D develops production judgment for containerized and event-driven application platforms on Google Cloud. The objective is not to deploy a sample and stop. It is to build a trustworthy software supply chain, select the right runtime, design identity and networking, control traffic and concurrency, engineer asynchronous delivery, and prove recovery under failure.

The chapters progress from artifact integrity to Kubernetes, serverless containers, functions, APIs, messaging, event routing, and workflow orchestration. Each lab requires measurable evidence, least-privilege identities, controlled rollout, observability, failure testing, and cleanup.

Volume 8 Progress

Part	Coverage	Status
8A	Foundations: hierarchy, IAM, tools, compute, VPC, billing	Complete
8B	Core infrastructure: fleets, networking, delivery, storage, connectivity, DR	Complete
8C	Databases, analytics, data engineering, AI-ready platforms, migration	Complete
8D	Containers, serverless, APIs, events, and workflows	This book
8E	Terraform, CI/CD, observability, security, governance, FinOps	Next
8F	Enterprise projects, incidents, certifications, interviews, final capstone	Planned

Platform Safety and Cost Controls

- Use an isolated project, budgets, quotas, small clusters, short-lived revisions, and explicit deletion steps. GKE nodes, load balancers, NAT, Artifact Registry storage, Apigee, and networking resources can continue charging after a lab ends.
- Never download long-lived service-account keys for runtime workloads. Prefer Workload Identity Federation for GKE, attached service identities, and short-lived federation for CI/CD.
- Use immutable image digests, vulnerability evidence, staged promotion, policy enforcement, and rollback. A successful image build is not evidence that an artifact is authorized or safe to run.
- Assume at-least-once delivery unless a documented service guarantee and end-to-end design prove otherwise. Consumers must be idempotent and poison messages must be isolated.
- Distinguish synchronous request paths, asynchronous queues, event routing, scheduled triggers, and durable orchestration. Selecting the wrong primitive creates reliability and cost problems.

Table of Contents

Chapter	Title
31	Artifact Registry and Container Supply-Chain Engineering
32	Google Kubernetes Engine Architecture and Cluster Design

Chapter	Title
33	GKE Production Operations, Networking, Scaling, and Fleets
34	Cloud Run Services, Jobs, and Worker Pools
35	Cloud Run Functions and Event-Driven Compute
36	API Gateway, Apigee, and API Platform Engineering
37	Pub/Sub Messaging and Delivery Engineering
38	Eventarc Event Routing and Event-Mesh Engineering
39	Workflows, Cloud Tasks, and Cloud Scheduler
40	Enterprise Cloud-Native Architecture and Capstone

Chapter 31 - Artifact Registry and Container Supply-Chain Engineering

Building immutable, attestable, and policy-controlled software delivery

Learning Objectives

- Design Artifact Registry repositories, identities, retention, and regional placement.
- Build and promote images by digest instead of mutable tags.
- Integrate vulnerability scanning, provenance, SBOMs, signing, and Binary Authorization.
- Operate artifact lifecycle, incident response, and rollback.

31.1 Repository Architecture and Trust Boundaries

Artifact Registry stores container images and language packages in regional or multi-regional repositories. Repository boundaries should reflect ownership, environment, residency, promotion, and access. Avoid one global writeable repository shared by every team and environment.

Pattern	Strength	Risk/Control
Repository per environment	Clear promotion boundary	More administration; automate policy and retention
Repository per product/team	Strong ownership	Cross-team sharing requires deliberate IAM
Central build plus regional mirrors	Controlled source and locality	Replication/promotion evidence must be monitored
Single shared repository	Simple start	Broad permissions and tag collision; unsuitable for mature production

31.2 Immutable Build and Promotion

Tags are convenient human references but can move. Production deployment records should resolve to immutable digests. Build once, test once, and promote the same digest through environments. Rebuilding from the same source can produce different bytes because dependencies, base images, timestamps, and build systems change.

- Pin base images and critical dependencies.
- Record source revision, builder identity, build parameters, digest, test evidence, and approvals.
- Separate build identity from deploy identity.
- Require pull-only access for runtimes and restricted write access for trusted pipelines.

```
gcloud artifacts repositories list --location=REGION
gcloud artifacts docker images list REGION-docker.pkg.dev/PROJECT/platform --include-tags
docker pull REGION-docker.pkg.dev/PROJECT/platform/orders@sha256:DIGEST
```

31.3 Vulnerability, Provenance, and Policy

Container Analysis and related security tooling can surface vulnerabilities and metadata. Supply-chain controls should combine scanning, provenance, software bill of materials, signatures or attestations, and deployment policy. Binary Authorization can prevent untrusted images from reaching GKE and supported deployment targets.

Control	Question Answered
Vulnerability scan	What known vulnerable components exist?
SBOM	What components and versions are included?
Provenance/attestation	Who built this artifact from which source and process?
Signature	Has an approved identity signed this digest?
Binary Authorization	Is this artifact permitted to deploy here?
Admission policy	Does the workload meet platform runtime requirements?

31.4 Lifecycle and Incident Response

Retention policies, cleanup rules, tag immutability conventions, and access logs prevent repositories from becoming uncontrolled archives. During a supply-chain incident, identify affected digests, stop promotion, revoke compromised identities, deny deployment, roll back to a known-good digest, rebuild from trusted inputs, and preserve evidence.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Image pulls fail in runtime	Repository IAM, VPC/perimeter restriction, or wrong image path	Check workload identity, repository location, digest, audit logs, and network/perimeter policy.
Scan shows old critical CVE after rebuild	Base layer remains vulnerable or cached	Inspect SBOM/layers and builder cache; update pinned base and rebuild with provenance.
Binary Authorization blocks release	Missing/invalid attestation or policy mismatch	Inspect policy and attestor identity; create evidence through the trusted pipeline, never bypass casually.
Storage cost grows continuously	No cleanup or duplicated tags/artifacts	Measure repository bytes and age; enforce retention while preserving regulated release evidence.

Hands-on Lab 31: Build a Policy-Controlled Image Promotion Path

Prerequisites

- Artifact Registry, Cloud Build or another trusted builder, a small test runtime, and permissions for repository and policy configuration.

Implementation

- Create separate development and production repositories with least-privilege writer and reader roles.
- Build a small application from a pinned base image and record source revision and digest.
- Generate vulnerability and SBOM evidence; classify findings by exploitability and runtime exposure.
- Promote the exact digest to production without rebuilding it.
- Create an attestation or signing step and configure a policy that rejects an unsigned test image.

6. Deploy the approved digest and demonstrate rollback to the prior digest.
7. Configure cleanup rules and export audit evidence.

Validation Evidence

- The same digest is traceable across build, test, promotion, and deployment.
- An unapproved artifact is denied while an approved artifact deploys.
- Runtime identity can pull but cannot overwrite images.
- Rollback and repository cleanup are proven.

Cleanup

- Delete test images, repositories if dedicated, policy objects, service accounts, builds, and runtime resources.

Interview Questions and Model Answers

1. Why deploy by digest?

A digest identifies immutable content; a tag can be moved and does not by itself prove which bytes were deployed.

2. Does a vulnerability scan prove an image is safe?

No. It covers known findings in scanned components and must be combined with provenance, configuration, exposure analysis, and policy.

3. What is Binary Authorization?

A deploy-time policy control that evaluates attestations and rules before allowing an image to run.

4. Why separate build and deploy identities?

It limits blast radius and prevents one compromised identity from both creating and releasing artifacts.

Chapter Summary

- Artifact integrity begins before deployment.
- Build once and promote immutable digests.
- Scanning, provenance, signing, and policy answer different trust questions.
- Repository lifecycle and incident response require operational ownership.

Chapter 32 - Google Kubernetes Engine Architecture and Cluster Design

Selecting GKE modes, designing cluster boundaries, and securing workload identity

Learning Objectives

- Compare GKE Autopilot and Standard operating models.
- Design cluster, project, region, release-channel, and tenancy boundaries.
- Use Workload Identity Federation for GKE and least-privilege Kubernetes authorization.
- Plan node, control-plane, upgrade, and availability architecture.

32.1 Autopilot and Standard

GKE Autopilot is a mode where Google manages node infrastructure configuration, scaling, security defaults, and many operational settings. Standard provides deeper control of nodes, system configuration, and specialized workloads. Modern GKE also allows selected Autopilot-mode workloads in Standard clusters through ComputeClasses, but platform teams should adopt such features only after compatibility and lifecycle review.

Decision Area	Autopilot	Standard
Node operations	Google-managed	Platform-managed node pools and configuration
Security defaults	Stronger managed baseline	More flexibility and more responsibility
Specialized hardware/config	Supported set with managed constraints	Broad control for GPUs, local SSD, daemon workloads, custom settings
Cost model	Workload resource based for most pods	Node capacity plus cluster operations
Best fit	Most application workloads and small platform teams	Specialized or deeply customized platform requirements

32.2 Cluster and Tenancy Boundaries

A cluster is a security, availability, quota, upgrade, and blast-radius boundary. Separate clusters when workloads require incompatible trust, lifecycle, networking, compliance, or operational ownership. Namespaces improve organization and policy but do not automatically provide hard multi-tenancy.

- Prefer regional clusters for production availability where cost and service support allow.
- Use separate projects and folders to reinforce billing, IAM, quota, and policy boundaries.
- Choose release channels and maintenance windows deliberately.
- Document which teams own cluster, namespaces, platform add-ons, and application resources.

32.3 Identity and Authorization

Workload Identity Federation for GKE is the recommended way for workloads to access Google Cloud APIs without downloaded service-account keys. Autopilot enables it by default; Standard requires cluster and node-pool configuration. Kubernetes RBAC controls Kubernetes API actions, while Google Cloud IAM controls Google Cloud resources. Both must be least-privileged.

```
gcloud container clusters describe CLUSTER --region=REGION --
format='yaml(autopilot,releaseChannel,workloadIdentityConfig,privateClusterConfig)'
kubectl auth can-i list secrets --as=system:serviceaccount:orders:orders-api
```

32.4 Availability and Upgrade Design

Control-plane availability does not guarantee application availability. Applications need topology spread, multiple replicas, PodDisruptionBudgets, readiness probes, graceful termination, suitable storage, and dependency resilience. Release channels, surge upgrades, maintenance exclusions, and version-skew rules must be integrated with application testing.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Pods cannot call Google API	Workload identity binding, IAM role, audience, or metadata issue	Inspect Kubernetes service account annotation/principal, token, IAM policy, and workload logs.
Cluster upgrade disrupts service	Single replica, strict PDB, slow termination, or no surge capacity	Review eviction events, PDB, topology, readiness, and node-pool upgrade settings.
Autopilot rejects workload	Unsupported privileged setting, host access, or resource request	Read admission error; redesign workload or justify Standard mode.
User can access unexpected namespaces	Overbroad Kubernetes RBAC or Google group mapping	Use kubectl auth can-i, RoleBinding review, audit logs, and group membership evidence.

Hands-on Lab 32: Create a Secure Regional GKE Foundation

Prerequisites

- A learning project, VPC/subnet, Artifact Registry image, and permissions to create a small GKE cluster.

Implementation

- Write an architecture decision comparing Autopilot and Standard for the workload.
- Create a regional Autopilot cluster or a minimal regional Standard cluster with a release channel.
- Create namespaces for platform and application ownership.
- Configure a Kubernetes service account to use Workload Identity Federation for GKE.
- Grant only the Google Cloud role required to read one test resource.
- Deploy a two-replica application with readiness, liveness, resource requests, topology spread, and a PDB.
- Test Kubernetes RBAC denial and workload access to the approved Google API.

Validation Evidence

- The cluster mode decision is evidence-based.
- No service-account key file is present.
- The workload accesses only the intended Google Cloud resource.
- A pod or node disruption does not remove all ready replicas.

Cleanup

- Delete workloads, IAM bindings, service accounts, cluster, network resources, and test artifacts.

Interview Questions and Model Answers

1. When is Autopilot a strong default?

When workloads fit supported constraints and the team benefits from Google-managed nodes and stronger managed defaults.

2. Why are namespaces not complete tenant isolation?

Cluster-level resources, kernel/node sharing, privileged controls, and administration remain shared.

3. What does Workload Identity Federation for GKE solve?

It gives Kubernetes workloads short-lived, identity-based access to Google Cloud APIs without static keys.

4. Why use a regional cluster?

It distributes the control plane and, with suitable node placement, improves resilience to a zonal failure.

Chapter Summary

- Choose GKE mode from workload and operating-model evidence.
- Treat cluster boundaries as security and failure boundaries.
- Kubernetes RBAC and Google Cloud IAM are complementary.
- Application availability must be engineered above the control plane.

Chapter 33 - GKE Production Operations, Networking, Scaling, and Fleets

Operating Kubernetes as a controlled multi-team production platform

Learning Objectives

- Engineer GKE networking, ingress, Gateway API, DNS, and network policy.
- Design horizontal, vertical, cluster, and node autoscaling with failure-state capacity.
- Operate upgrades, observability, backup, policy, and fleet-level governance.
- Troubleshoot Kubernetes systematically from user symptom to infrastructure evidence.

33.1 Networking and Application Delivery

GKE networking spans pod and service address allocation, VPC routing, load balancing, DNS, egress, private access, and policy. Gateway API provides role-oriented traffic management and is increasingly preferred for advanced L7 designs, while Ingress remains common. Network Policy controls pod traffic but must be paired with VPC firewall and identity controls.

Layer	Primary Control
External entry	Global/regional load balancer, Gateway/Ingress, TLS, Cloud Armor
Service discovery	Cluster DNS, Services, optional multi-cluster service discovery
East-west traffic	Services, Network Policy, optional Cloud Service Mesh
Google API access	Workload Identity Federation, Private Google Access/PSC where required
Internet egress	Cloud NAT, firewall policy, DNS controls, proxy where governed

33.2 Scaling and Capacity

Horizontal Pod Autoscaling reacts to workload metrics; Vertical Pod Autoscaling recommends or adjusts resource requests; cluster and node-pool autoscaling supply capacity. Scaling loops have delays and can conflict. Define minimum capacity, maximum spend, startup time, queue backpressure, and failure-state requirements.

- Set realistic requests because scheduling and autoscaling depend on them.
- Protect dependencies with concurrency and queue limits.
- Keep capacity for upgrades and zonal loss.
- Load test scale-up and scale-down, not only steady state.

33.3 Fleet and Multi-Cluster Operations

GKE fleets group clusters for consistent configuration, identity, policy, service mesh, and multi-cluster capabilities. A fleet is not a substitute for clear ownership or DR design. Standardize registration, configuration sync, policy, release channels, and observability while preserving justified exceptions.

Capability	Operational Goal
Fleet registration	Inventory and consistent identity boundary
Config Sync/GitOps	Versioned cluster and namespace configuration

Capability	Operational Goal
Policy Controller	Admission and compliance guardrails
Cloud Service Mesh	Traffic security and telemetry where justified
Multi-cluster services/gateways	Cross-cluster discovery and application delivery
Backup for GKE	Restore Kubernetes resources and supported persistent data

33.4 Incident and Upgrade Operations

Troubleshoot from the user-visible SLI through load balancer, Gateway/Ingress, Service endpoints, pod readiness, application logs, node pressure, control-plane events, and dependencies. During upgrades, use canary clusters or node pools for material changes, inspect deprecations, preserve rollback options, and communicate change windows.

```
kubectl get events -A --sort-by=.lastTimestamp
kubectl get pods -A -o wide
kubectl describe pod POD -n NAMESPACE
gcloud container operations list --region=REGION
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Service returns 503	No ready endpoints, unhealthy backend, policy, or app failure	Trace load balancer health to Gateway/Service/EndpointSlice and pod readiness.
Pods remain Pending	Insufficient resources, affinity, taint, quota, or IP exhaustion	Inspect scheduler events, requests, node pools, subnet secondary ranges, and quotas.
Autoscaling oscillates	Noisy metric, bad target, startup delay, or conflicting autoscalers	Graph metrics and replica decisions; stabilize target, windows, requests, and bounds.
Cross-cluster routing is partial	Fleet membership, service export, health, DNS, or policy mismatch	Inspect fleet status, exported services, gateway backends, health, and project/network IAM.

Hands-on Lab 33: Operate a Production-Style GKE Service

Prerequisites

- A small GKE cluster, Artifact Registry image, metrics, and permissions for load balancing and policy.

Implementation

- Deploy a versioned application using a Deployment, Service, Gateway or Ingress, and managed TLS where practical.
- Add Network Policy, PodDisruptionBudget, topology spread, resource requests, and graceful shutdown.
- Configure HPA with a measurable target and define dependency-safe maximum replicas.
- Generate load and capture scale-up latency, errors, pod scheduling, and dependency behavior.

5. Perform a rolling update and a failed canary; demonstrate rollback.
6. Simulate one failure: pod crash, node drain, denied network path, or bad readiness probe.
7. Create a fleet/policy/backup design for scaling this platform across clusters.

Validation Evidence

- Traffic reaches only healthy endpoints.
- Scaling evidence includes application and dependency metrics.
- The failed release is detected and rolled back.
- The incident timeline links symptom to Kubernetes and infrastructure evidence.

Cleanup

- Delete application, gateway/load balancer, policies, backup objects, and cluster if dedicated.

Interview Questions and Model Answers

1. Why can HPA fail even when configured correctly?

Metrics, resource requests, pod startup time, scheduler capacity, quotas, and dependency limits can prevent safe scaling.

2. What is the purpose of a PDB?

It limits voluntary disruption of available replicas; it does not guarantee availability during every failure.

3. Why use Gateway API?

It separates infrastructure and application roles and supports richer, portable traffic-management resources.

4. What does a GKE fleet provide?

A logical grouping for consistent identity, policy, configuration, service, and multi-cluster management capabilities.

Chapter Summary

- Production GKE operations cross Kubernetes and Google Cloud layers.
- Autoscaling requires dependency and cost guardrails.
- Fleets standardize multi-cluster governance but do not replace architecture.
- Troubleshooting should follow the request path and evidence chain.

Chapter 34 - Cloud Run Services, Jobs, and Worker Pools

Running containers without managing Kubernetes while preserving production controls

Learning Objectives

- Select Cloud Run services, jobs, or worker pools from execution semantics.
- Engineer revisions, traffic splitting, concurrency, autoscaling, and rollback.
- Design private networking, service identity, secrets, and authenticated invocation.
- Operate cost, cold starts, background work, and multi-region resilience.

34.1 Resource Models

Cloud Run is a fully managed application platform for code, functions, and containers. Services expose request-driven endpoints and autoscale. Jobs run tasks to completion. Worker pools run continuous background work without a load-balanced URL and, according to current documentation, do not provide automatic scaling. Choose from lifecycle semantics, not deployment convenience.

Resource	Use Case	Key Constraint
Service	HTTP/gRPC request handling, webhooks, APIs	Request lifecycle, configured concurrency and autoscaling
Job	Batch, migration, scheduled processing	Finite tasks with retries and completion state
Worker pool	Continuous pull consumers or background processors	No load-balanced URL; capacity managed explicitly
Function on Cloud Run	Single-purpose event/HTTP handler	Function framework/runtime conventions

34.2 Revisions, Traffic, and Scaling

Every configuration or image change creates a revision. Split traffic between revisions for canaries, migrate gradually, and retain known-good revisions for rollback. Concurrency determines how many requests one instance can serve; minimum instances reduce cold-start risk; maximum instances protect cost and downstream systems.

```
gcloud run deploy orders --image=REGION-docker.pkg.dev/PROJECT/apps/orders@sha256:DIGEST --region=REGION --no-allow-unauthenticated
gcloud run services update-traffic orders --region=REGION --to-revisions=orders-00042=10,orders-00041=90
```

34.3 Identity, Ingress, and VPC Connectivity

Use a dedicated service identity with least privilege. Keep services private unless public access is an explicit requirement. Ingress settings control accepted network paths; IAM controls who can invoke. Direct VPC egress lets services and jobs reach VPC resources without managing a Serverless VPC Access connector, and network tags can support granular firewall rules. Worker pools can use Direct VPC ingress and egress.

- Use Secret Manager references rather than plaintext environment variables for secrets.
- Use authenticated service-to-service invocation with identity tokens.
- Separate public frontend, internal API, batch, and worker identities.
- Control egress and DNS so private dependencies are reachable without broad internet access.

34.4 Reliability, Cost, and Multi-Region Design

Cloud Run services are regional. Multi-region designs use independent services, global application delivery, health-based routing, replicated state, and tested failover. Scale-to-zero saves cost but can add latency. Minimum instances, CPU allocation, concurrency, timeouts, and retries must be tuned together. Never let automatic scale overwhelm a database or third-party API.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Revision receives no traffic	Traffic assignment, tag, IAM, or failed startup	Inspect service/revision status, traffic percentages, container logs, port, and health.
Latency spikes after idle	Cold start, dependency connection setup, or image size	Measure startup; reduce image/dependency work, set minimum instances, and reuse connections.
Database connections exhausted	High instance count times per-instance pool	Cap Cloud Run maximum instances/concurrency and bound pool size; add backpressure.
Private dependency unreachable	Wrong Direct VPC egress subnet, firewall, DNS, or route	Inspect revision network config, network tags, firewall logs, DNS, and return path.

Hands-on Lab 34: Release a Private Cloud Run Application Safely

Prerequisites

- Artifact Registry image, VPC/subnet, a private dependency or mock, and IAM permissions for Cloud Run.

Implementation

1. Deploy a private Cloud Run service by immutable digest with a dedicated service account.
2. Configure authenticated invocation and prove anonymous access is denied.
3. Connect to a VPC resource using Direct VPC egress and a least-privilege firewall rule.
4. Set concurrency, minimum and maximum instances, timeout, and request limits from a capacity assumption.
5. Deploy a second revision, send 10 percent canary traffic, and observe latency/errors.
6. Inject a startup or application failure and roll traffic back.
7. Create a Cloud Run job for a finite reconciliation task and define when a worker pool would be preferable.

Validation Evidence

- Only authorized identities invoke the service.
- Private networking works without broad ingress.
- Canary and rollback evidence identifies revisions by digest.
- Scaling limits protect the dependency.

Cleanup

- Delete services, jobs, worker pools if created, VPC rules, service accounts, and images.

Interview Questions and Model Answers

1. How do services and jobs differ?

Services handle requests and autoscale; jobs execute finite tasks to completion with task-level retries.

2. Why set maximum instances?

To bound cost and protect downstream systems from unbounded autoscaling.

3. What is a Cloud Run revision?

An immutable version of a service configuration and container image used for traffic management and rollback.

4. When use a worker pool?

For continuous background processing without a public endpoint when explicitly managed capacity is acceptable.

Chapter Summary

- Cloud Run offers distinct service, job, worker, and function execution models.
- Revisions make gradual delivery and rollback first-class.
- IAM, ingress, and VPC controls are separate layers.
- Autoscaling must be bounded by dependency capacity and cost.

Chapter 35 - Cloud Run Functions and Event-Driven Compute

Engineering small event handlers with production reliability and lifecycle controls

Learning Objectives

- Use current Cloud Run functions terminology and deployment models.
- Design HTTP and CloudEvents handlers with idempotency and bounded retries.
- Secure function identity, ingress, secrets, and dependencies.
- Choose functions, Cloud Run services, jobs, or Workflows appropriately.

35.1 Current Function Model

Cloud Functions (2nd gen) is now Cloud Run functions. Existing Cloud Functions v2 APIs and gcloud functions workflows remain available for compatibility, but Google recommends transitioning to functions in Cloud Run. Functions use language frameworks and can receive HTTP requests or CloudEvents from supported event sources.

Choice	Strength	When to Reconsider
Cloud Run function	Minimal single-purpose handler	Complex container/runtime control or many endpoints
Cloud Run service	Full container and request lifecycle	Only a tiny handler is required
Cloud Run job	Finite asynchronous batch	Immediate request/event response is required
Workflows	Durable orchestration of services/APIs	Heavy compute or custom runtime logic

35.2 Idempotency and Event Semantics

Events can be redelivered. Use a stable event or business key, store processing state atomically, and make side effects conditional. Acknowledging before durable completion loses work; retrying non-idempotent effects duplicates work. Poison events need bounded retry and dead-letter or quarantine handling.

- Validate event type, source, subject, schema, and size.
- Record event ID and processing outcome with expiration where appropriate.
- Use transactional outbox/inbox or conditional writes for critical effects.
- Separate transient errors from permanent validation failures.

35.3 Security and Dependency Controls

Each function needs a dedicated runtime identity. Restrict invocation and ingress, use Secret Manager, and avoid embedding credentials. Cap instances and concurrency to protect databases. Network private dependencies through supported Cloud Run networking, and avoid unrestricted egress for sensitive handlers.

```
gcloud run functions list --region=REGION
gcloud functions list --regions=REGION
gcloud run services describe FUNCTION_SERVICE --region=REGION
```

35.4 Observability and Lifecycle

Measure invocation count, latency, errors, retries, cold starts, dependency calls, and business outcomes. Structured logs should include trace and event identifiers without sensitive payloads. Runtime versions, dependencies, event providers, and trigger APIs evolve; maintain upgrade and deprecation ownership.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Function processes event twice	At-least-once delivery and no idempotency record	Use event/business key and atomic deduplication before side effects.
Function retries forever or floods logs	Transient/permanent errors not separated or retry policy too broad	Classify errors, bound retries, quarantine invalid events, and alert on age.
HTTP invocation is denied	Invoker IAM, audience, token, or ingress mismatch	Inspect IAM policy, identity token audience, ingress path, and audit logs.
Deploy succeeds but trigger does not fire	Eventarc trigger, region, service account, filter, or provider permission	Inspect trigger state, audit logs, provider event, filters, and destination logs.

Hands-on Lab 35: Build an Idempotent Cloud Run Function

Prerequisites

- A storage bucket or Pub/Sub topic, Firestore/another state store for deduplication, and permissions for Cloud Run functions and Eventarc.

Implementation

1. Create a small CloudEvents function that validates event metadata and payload.
2. Deploy it with a dedicated runtime service account and no unnecessary public access.
3. Record event IDs and processing status atomically before a simulated external effect.
4. Publish the same event twice and prove the effect occurs once.
5. Inject a transient dependency failure and observe bounded retry.
6. Inject a permanent invalid event and route it to quarantine or record a terminal failure.
7. Create dashboards or log queries for latency, retries, duplicate suppression, and business outcome.

Validation Evidence

- Duplicate delivery does not duplicate the business effect.
- Runtime identity has only required roles.
- Transient and permanent failures have different outcomes.
- Logs correlate trigger, event, function execution, and effect.

Cleanup

- Delete function/service, trigger, topic/bucket, state records, service accounts, and test artifacts.

Interview Questions and Model Answers

1. Why are functions not automatically exactly once?

The trigger and downstream side effects can be retried or partially fail; end-to-end idempotency is still required.

2. What changed in product naming?

Cloud Functions (2nd gen) is now Cloud Run functions; compatibility APIs remain, while new guidance points to Cloud Run.

3. When choose a Cloud Run service instead?

When you need full container control, multiple routes, custom servers, or more complex request behavior.

4. How should permanent errors be handled?

Do not retry indefinitely; record/quarantine them with enough evidence for correction and replay.

Chapter Summary

- Cloud Run functions are lightweight Cloud Run-based handlers.
- Idempotency is the central event-processing requirement.
- Identity, ingress, and scaling limits protect dependencies.
- Runtime and trigger lifecycle need active ownership.

Chapter 36 - API Gateway, Apigee, and API Platform Engineering

Treating APIs as governed products rather than unmanaged endpoints

Learning Objectives

- Select API Gateway or Apigee from lifecycle, policy, analytics, and hybrid requirements.
- Design API contracts, authentication, quotas, versioning, and error behavior.
- Operate gateways with private backends, observability, rollout, and rollback.
- Establish API product ownership and consumer governance.

36.1 API Platform Selection

API Gateway provides a managed front door for well-defined REST APIs backed by services such as Cloud Run. Apigee is Google Cloud's enterprise API management platform for proxies, products, developer onboarding, advanced policies, analytics, monetization, and hybrid deployment models. The right choice depends on API program maturity, not traffic alone.

Requirement	API Gateway	Apigee
Managed REST front door	Strong fit	Supported but may be more platform than needed
Advanced policy and mediation	Basic contract/auth/routing use cases	Rich policy, transformation, threat, quota, caching
Developer portal/API products	Limited	Core capability
Hybrid or multi-cloud gateway	Not primary design	Apigee hybrid supports distributed runtime
Enterprise analytics/lifecycle	Basic operational data	Deep API analytics and product governance

36.2 Contract and Consumer Design

An API is a contract. Use OpenAPI or another governed specification, semantic versioning, predictable errors, idempotency keys, pagination, timeouts, and compatibility rules. Separate internal, partner, and public products because identity, support, quotas, and change expectations differ.

- Authenticate users and workloads with the appropriate identity protocol.
- Authorize at both gateway and backend; do not trust headers from untrusted clients.
- Use quotas and spike arrest to protect products and dependencies.
- Avoid leaking backend errors or sensitive implementation details.

36.3 Backend Security and Release

The gateway should invoke private backends using a dedicated identity. Deploy API configuration and proxy revisions through version control and staged environments. Test policy ordering, token validation, CORS, payload limits, timeout alignment, and rollback. A gateway is not a substitute for backend authorization.

```
gcloud api-gateway gateways list --location=REGION
gcloud api-gateway api-configs list --api=orders-api
gcloud run services get-iam-policy orders --region=REGION
```

36.4 API Operations and Product Metrics

Monitor availability, p50/p95/p99 latency, 4xx/5xx by consumer and operation, quota rejection, backend latency, token failures, and business transactions. API product owners need deprecation notices, support objectives, consumer inventory, and incident communication.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Gateway returns 401/403	Token issuer/audience, API key, IAM, or backend authorization	Trace gateway auth decision, token claims, invoker role, and backend audit logs.
Gateway 5xx but backend looks healthy	Timeout mismatch, policy error, payload transformation, or DNS/network issue	Compare gateway and backend traces; inspect config/proxy revision and backend latency.
One consumer degrades everyone	No quota, shared credential, or expensive endpoint	Identify consumer key/identity; add product quotas, caching, pagination, or async design.
New API config has no effect	Gateway still references old config or rollout incomplete	Describe gateway/config, verify revision, propagate, test canary hostname, and roll back if needed.

Hands-on Lab 36: Publish and Protect a Cloud Run API

Prerequisites

- A private Cloud Run backend, an OpenAPI contract, and permissions for API Gateway; Apigee optional for comparison.

Implementation

1. Define an OpenAPI contract with operations, schemas, errors, authentication, and versioning.
2. Deploy the backend privately with a dedicated service identity.
3. Create an API Gateway API, config, and gateway that authenticates callers and invokes the backend.
4. Add request limits, timeout behavior, and backend authorization.
5. Run positive, invalid-token, over-limit, malformed-payload, and backend-timeout tests.
6. Deploy a backward-compatible second config and rehearse rollback.
7. Write an architecture decision describing when the organization should graduate to Apigee.

Validation Evidence

- The backend is not directly exposed to unapproved callers.
- Contract and runtime behavior match for success and error cases.
- Gateway and backend logs correlate the request.
- Rollback restores the prior API behavior.

Cleanup

- Delete gateway, API configs, backend, service accounts, DNS if used, and test credentials.

Interview Questions and Model Answers

1. When is API Gateway sufficient?

For a managed REST front door with contract-based routing, authentication, and simpler API governance.

2. Why use Apigee?

For enterprise API products, rich policies, developer ecosystem, analytics, and hybrid or multi-environment management.

3. Should backend authorization remain?

Yes. Gateway authentication is one layer; the backend must still enforce trusted identity and resource authorization.

4. What is an API product?

A managed package of API operations, access terms, quotas, consumers, lifecycle, support, and business ownership.

Chapter Summary

- API management is a product and governance discipline.
- API Gateway and Apigee solve different maturity levels.
- Gateway and backend authorization must reinforce each other.
- Contracts, telemetry, quotas, and deprecation are production requirements.

Chapter 37 - Pub/Sub Messaging and Delivery Engineering

Designing asynchronous systems for replay, duplicates, ordering, and backpressure

Learning Objectives

- Model topics, subscriptions, publishers, and subscriber delivery types.
- Engineer idempotency, ordering, exactly-once options, dead-lettering, and replay.
- Control flow, backlog, retention, security, and regional reliability.
- Operate message-driven systems with business and platform telemetry.

37.1 Messaging Model and Subscription Types

Publishers send messages to topics; subscriptions define independent delivery state for consumers. Pull gives subscribers flow control; push sends authenticated HTTP requests; BigQuery and Cloud Storage subscriptions can deliver directly to supported sinks. Decouple producer event contracts from consumer processing state.

Subscription	Best Fit	Primary Control
Pull/streaming pull	High-throughput workers	Client flow control, leases, concurrency
Push	HTTP service or serverless consumer	Endpoint auth, response codes, max instances
BigQuery	Direct analytical ingestion	Schema/table mapping and permissions
Cloud Storage	Batch/object delivery	File size/time thresholds and downstream processing

37.2 Delivery Semantics and Ordering

At-least-once delivery is the safe default assumption. Exactly-once delivery helps subscribers determine successful acknowledgments and prevent redelivery after a successful ack, but guarantees are regional and do not automatically make downstream side effects exactly once. Ordering keys preserve order for related messages within documented regional semantics and reduce parallelism for a hot key.

- Use idempotency keys and transactional state.
- Acknowledge only after durable processing.
- Use ordering only when business correctness requires it.
- Keep messages small; store large payloads in durable storage and publish references.

37.3 Dead Letters, Retry, Retention, and Replay

Subscriptions can use retry policies and dead-letter topics. Dead-letter forwarding is best-effort and depends on correct service-agent IAM. Message retention, topic retention, snapshots, and seek support recovery and replay. Replaying can reproduce side effects, so consumer idempotency and change controls remain mandatory.

```
gcloud pubsub topics create orders
gcloud pubsub subscriptions create orders-worker --topic=orders --dead-letter-topic=orders-dlq --max-delivery-attempts=10
gcloud pubsub subscriptions describe orders-worker
```

37.4 Flow Control, Reliability, and Observability

Track oldest unacked message age, backlog bytes/messages, delivery latency, nack/expiration, dead-letter rate, publisher errors, quota, and business completion. Flow control prevents a subscriber from accepting more work

than it can process. Cross-region failover designs must account for regional ordering/exactly-once scope and duplicate publication.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Backlog grows while CPU is low	Blocked dependency, one hot ordering key, flow limit, or ack issue	Inspect oldest age, per-key distribution, dependency latency, ack deadline, and client logs.
Messages appear multiple times	Expected redelivery, failed ack, retry, or publisher duplicate	Correlate message ID and business key; enforce idempotency and ack after durable completion.
Dead-letter topic remains empty	Missing Pub/Sub service-agent roles or misconfigured subscription	Check subscription dead-letter settings, service-agent IAM, attempts, and active consumers.
Push subscription returns 403	OIDC token audience, invoker IAM, or endpoint ingress	Inspect push auth config, token claims, Cloud Run invoker policy, and endpoint logs.

Hands-on Lab 37: Build an Idempotent Pub/Sub Processing Pipeline

Prerequisites

- Pub/Sub, a Cloud Run service or worker, and Firestore/SQL for processing state.

Implementation

1. Create an orders topic, primary subscription, dead-letter topic, and dead-letter subscription.
2. Implement a subscriber that uses a business event ID and atomic processing record.
3. Publish normal, duplicate, malformed, and dependency-failure messages.
4. Configure bounded retry and prove malformed messages reach dead-letter handling.
5. Create a snapshot, process messages, then seek/replay in a controlled test.
6. Apply flow control and measure backlog, oldest age, processing throughput, and dependency latency.
7. Document when ordered delivery or exactly-once subscription mode would be justified.

Validation Evidence

- Duplicate messages create one business outcome.
- Poison messages are isolated and diagnosable.
- Replay does not duplicate completed effects.
- Backlog alerts identify capacity or dependency failure.

Cleanup

- Delete subscriptions, topics, services, state records, service accounts, and dashboards created only for the lab.

Interview Questions and Model Answers

1. Does Pub/Sub exactly-once make the whole business transaction exactly once?

No. It improves delivery/ack semantics; downstream side effects still require idempotency or transactions.

2. Why is a topic not a queue?

Each subscription has independent delivery state, allowing multiple consumers to receive the same published message.

3. What does oldest unacked age show?

How far the slowest unprocessed work is falling behind, often a stronger SLO signal than message count.

4. Why can ordering reduce throughput?

Messages sharing an ordering key must be delivered in sequence, limiting parallelism for that key.

Chapter Summary

- Subscriptions isolate consumer delivery state.
- At-least-once processing requires idempotency.
- Dead letters and replay need correct IAM and operational runbooks.
- Backlog age, flow control, and dependency capacity drive reliability.

Chapter 38 - Eventarc Event Routing and Event-Mesh Engineering

Routing CloudEvents with centralized governance and distributed ownership

Learning Objectives

- Compare Eventarc Standard and Advanced.
- Design event sources, filters, destinations, identities, and CloudEvents contracts.
- Engineer retries, dead-lettering, observability, and regional placement.
- Establish event governance without recreating a centralized monolith.

38.1 Standard and Advanced Editions

Eventarc builds event-driven architectures without maintaining routing infrastructure. Eventarc Standard connects supported Google sources, Pub/Sub, and destinations with managed triggers. Eventarc Advanced adds event buses and pipelines for centralized governance, richer routing, transformation, and organization-scale eventing. Use Advanced when the governance and routing model justifies the additional platform layer.

Capability	Standard	Advanced
Primary resource	Trigger	Bus plus pipeline
Operating model	Application/team-level routing	Central policy with distributed integration logic
Transformation/routing	Source filters and destination routing	Richer content-aware processing and routing
Best fit	Direct event source to service/workflow	Enterprise event mesh and cross-team governance

38.2 CloudEvents Contract and Filtering

Eventarc delivers events in CloudEvents format. Govern type, source, subject, ID, time, data schema, and business extension attributes. Filters reduce unwanted delivery but should not replace consumer validation. Schema compatibility and ownership are necessary because independent producers and consumers evolve.

- Version event contracts and define additive versus breaking changes.
- Keep event payloads minimal and avoid sensitive data unless approved.
- Use stable business identifiers separate from transport message IDs.
- Document source and destination region compatibility.

38.3 Destination Security and Delivery

Each trigger or pipeline uses service identities and requires permission to receive source events and invoke the destination. Destinations should be private by default and grant invocation only to the routing identity. Retries can produce duplicate delivery, so destinations remain idempotent.

```
gcloud eventarc triggers list --location=REGION
gcloud eventarc triggers describe storage-finalized --location=REGION
gcloud run services get-iam-policy event-handler --region=REGION
```

38.4 Event Mesh Governance and Operations

An event catalog should record owner, purpose, schema, classification, retention, consumers, region, SLO, and deprecation. Platform teams govern buses, identity, observability, and policy; product teams own event

meaning and processing. Monitor delivery errors, retry age, destination latency, filtered volume, and business completeness.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Trigger is active but no events arrive	Filter mismatch, source audit/API config, region, service agent, or destination IAM	Inspect trigger conditions, source event/audit log, service-agent roles, and destination invocation logs.
Destination receives duplicates	Retry after timeout or at-least-once transport	Use CloudEvent/business ID and idempotent state; return success only after durable completion.
Events cross wrong trust boundary	Overbroad bus/pipeline, IAM, or classification	Review event catalog, routing policy, payload classification, and project/folder boundaries.
Advanced pipeline rejects/changes payload	Transformation or schema policy mismatch	Inspect pipeline stages and dead-letter/error records; version and test transformations.

Hands-on Lab 38: Route Governed Events to a Private Service

Prerequisites

- A Cloud Storage bucket or Pub/Sub source, Eventarc, a private Cloud Run destination, and a state store.

Implementation

1. Define an event contract with required CloudEvents and business attributes.
2. Deploy a private idempotent Cloud Run event handler.
3. Create an Eventarc Standard trigger with narrow source filters and a dedicated identity.
4. Generate matching and non-matching events; verify filtering and authenticated delivery.
5. Inject destination timeout and prove redelivery does not duplicate the business effect.
6. Create a dead-letter or error-handling design and an event-catalog entry.
7. Write an architecture decision for when Eventarc Advanced buses and pipelines would be adopted.

Validation Evidence

- Only matching events invoke the destination.
- The destination is not anonymously callable.
- Duplicate delivery is harmless.
- Event ownership, classification, SLO, and deprecation are documented.

Cleanup

- Delete trigger, service, bucket/topic, state records, IAM bindings, and test events.

Interview Questions and Model Answers

1. What is Eventarc Standard?

Managed trigger-based routing from supported event sources to services and workflows.

2. Why use Eventarc Advanced?

For organization-scale event buses and pipelines with centralized governance and richer distributed routing logic.

3. Why use CloudEvents?

It standardizes event metadata so producers, routers, and consumers can interoperate and govern events consistently.

4. Do filters eliminate consumer validation?

No. Consumers must still validate type, source, schema, authorization context, and business rules.

Chapter Summary

- Eventarc separates event routing from application code.
- Standard and Advanced support different governance scales.
- CloudEvents contracts require ownership and compatibility rules.
- Idempotency and authenticated destinations remain mandatory.

Chapter 39 - Workflows, Cloud Tasks, and Cloud Scheduler

Combining durable orchestration, controlled task delivery, and scheduled execution

Learning Objectives

- Select Workflows, Cloud Tasks, Cloud Scheduler, Pub/Sub, and jobs correctly.
- Build durable orchestration with connectors, retries, callbacks, and compensation.
- Control task queues, rate, retry, deduplication, and authenticated targets.
- Operate schedules and workflows with traceability and reconciliation.

39.1 Selecting the Coordination Primitive

Workflows orchestrates steps and API calls with durable execution state. Cloud Tasks delivers individual HTTP tasks with queue-level rate and retry controls. Cloud Scheduler triggers work on a schedule. Pub/Sub distributes events to independent consumers. Cloud Run jobs perform finite compute. Combining them intentionally is more reliable than forcing one product into every role.

Need	Best Starting Point
Durable multi-step orchestration	Workflows
Per-request asynchronous HTTP delivery	Cloud Tasks
Cron or calendar trigger	Cloud Scheduler
Fan-out event distribution	Pub/Sub/Eventarc
Finite parallel batch compute	Cloud Run jobs or managed batch platform
Human/external callback wait	Workflows callbacks

39.2 Workflows Design

Workflows definitions support calls, assignments, conditions, loops, parallel branches, subworkflows, retries, error handling, and returns. Google Cloud connectors simplify API calls and include behavior for retries and long-running operations. Design explicit compensation for operations that cannot be rolled back atomically.

- Use a dedicated workflow service account.
- Pass references, not large sensitive payloads.
- Define retry predicates only for transient and idempotent operations.
- Record business state outside workflow execution history when long-term reconciliation is required.

```
gcloud workflows deploy order-orchestrator --source=workflow.yaml --location=REGION --service-account=workflow-sa@PROJECT.iam.gserviceaccount.com
gcloud workflows run order-orchestrator --location=REGION --data='{"orderId":"o-123"}'
```

39.3 Cloud Tasks Queue Engineering

Cloud Tasks decouples request acceptance from controlled HTTP delivery. Queue dispatch rate and concurrent dispatch limits protect targets. Task names can support bounded deduplication, but business idempotency remains necessary. OIDC-authenticated requests let queues invoke private Cloud Run services.

Queue Control	Purpose
Dispatch rate	Limit requests per second

Queue Control	Purpose
Max concurrent dispatches	Bound in-flight pressure on target
Retry configuration	Control backoff, attempts, and age
Task schedule time	Delay delivery until a specific time
OIDC service account	Authenticate to private HTTP target
Task name	Support operational traceability and bounded duplicate prevention

39.4 Scheduling, Failure, and Reconciliation

Cloud Scheduler invokes HTTP, Pub/Sub, or supported targets on a cron schedule. Scheduled execution is not proof of business completion. Every scheduled process should be idempotent, observable, and reconcilable. For workflows, handle execution concurrency and backlogging deliberately; time-sensitive work may require backlogging to be disabled and alternative admission control.

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
Workflow stops after partial success	Unhandled error or no compensation path	Inspect execution step/error; execute compensation or reconciliation and update retry policy.
Tasks overload target	Dispatch/concurrency too high or target autoscaling unbounded	Reduce queue rate/concurrency, cap target instances, and measure dependency capacity.
Scheduled job runs twice	Retry, manual run, overlapping schedule, or clock assumptions	Use business-period idempotency key and lease/lock; reconcile outputs.
Workflow receives 429	Execution concurrency quota or downstream quota	Inspect execution/backlog state and quotas; throttle intake, enable/disable backlogging appropriately, or batch.

Hands-on Lab 39: Build a Durable Order Orchestration

Prerequisites

- Workflows, Cloud Tasks, Cloud Scheduler, two small private Cloud Run endpoints, and a state store.

Implementation

- Define an order state machine with reserve, charge, fulfill, and compensate outcomes.
- Implement reserve and fulfill endpoints as idempotent private Cloud Run services.
- Create a Workflows definition using connectors/HTTP calls, bounded retries, and compensation.
- Use Cloud Tasks to deliver one slow step with queue rate and OIDC authentication.
- Create a Cloud Scheduler reconciliation trigger for incomplete orders.
- Inject transient, permanent, duplicate, and timeout failures.
- Capture workflow execution, task attempt, service trace, and final business-state evidence.

Validation Evidence

- Transient failures retry without duplicate effects.
- Permanent failure executes compensation or terminal reconciliation.
- Queue limits protect the target.
- Scheduled reconciliation repairs or reports incomplete state.

Cleanup

- Delete workflows, queues, tasks, schedules, services, state records, service accounts, and test logs where appropriate.

Interview Questions and Model Answers

1. When use Workflows instead of Pub/Sub?

When a durable central sequence, branching, error handling, or compensation across services is required.

2. Why use Cloud Tasks?

To deliver individual HTTP work asynchronously with controlled rate, concurrency, scheduling, authentication, and retry.

3. Does Cloud Scheduler guarantee exactly one execution?

No. Scheduled targets must be idempotent and reconcilable.

4. What is compensation?

A business operation that semantically reverses or mitigates a completed step when a later step fails.

Chapter Summary

- Use orchestration, queues, events, schedules, and jobs for their intended semantics.
- Retries require idempotency and error classification.
- Queue limits are dependency-protection controls.
- Reconciliation closes gaps that distributed transactions cannot eliminate.

Chapter 40 - Enterprise Cloud-Native Application Architecture and Capstone

Integrating supply chain, GKE, Cloud Run, APIs, events, workflows, security, and operations

Learning Objectives

- Design a multi-runtime application platform with explicit service-selection decisions.
- Integrate identity, networking, API, messaging, eventing, orchestration, and data controls.
- Define SLOs, rollout, rollback, DR, incident response, and cost governance.
- Produce portfolio-grade architecture and validation evidence.

40.1 Reference Architecture

The capstone uses a global commerce platform: public API entry, private synchronous services, asynchronous order events, workflow-based fulfillment, scheduled reconciliation, and analytics export. GKE hosts specialized services requiring Kubernetes; Cloud Run hosts request-driven services and jobs; functions handle narrow events; Pub/Sub and Eventarc decouple producers; Workflows coordinates business steps.

Layer	Reference Control
Supply chain	Artifact Registry, immutable digests, scan/provenance, Binary Authorization
Runtime	GKE for Kubernetes needs; Cloud Run for managed containers; functions for narrow handlers
API	API Gateway or Apigee with private backends and product quotas
Async/event	Pub/Sub, Eventarc contracts, dead-letter, replay, idempotency
Orchestration	Workflows, Cloud Tasks, Scheduler reconciliation
Identity/network	Workload identities, private ingress, Direct VPC/GKE networking, least privilege
Operations	SLOs, tracing, dashboards, canaries, game days, cost allocation

40.2 Architecture Decision Records

Every service choice must include context, options, decision, consequences, security, cost, reversibility, and validation. Avoid choosing GKE because it is familiar or serverless because it is easy. Match runtime to operational control, workload shape, portability, latency, scaling, compliance, and team capability.

- ADR-1: GKE Autopilot versus Standard versus Cloud Run.
- ADR-2: API Gateway versus Apigee.
- ADR-3: Pub/Sub direct delivery versus Eventarc routing.
- ADR-4: choreography versus Workflows orchestration.
- ADR-5: regional active-passive versus active-active application design.

40.3 Reliability and Security Qualification

Qualification tests include image-policy denial, canary rollback, pod/node disruption, Cloud Run cold start and scale limit, duplicate event, dead-letter recovery, workflow compensation, identity denial, private network

failure, and regional dependency failover. Security evidence includes IAM, service identities, API authorization, artifact provenance, secret access, audit logs, and policy exceptions.

40.4 Operating Model and Portfolio Evidence

The final submission should contain diagrams, threat model, data flow, dependency inventory, SLOs, dashboards, runbooks, ADRs, pipeline evidence, load-test results, failure-test timeline, cost model, cleanup plan, and a concise executive briefing. Evidence should show decisions and outcomes, not screenshots without interpretation.

```
gcloud asset search-all-resources --scope=projects/PROJECT_ID
gcloud logging read 'severity>=ERROR' --project=PROJECT_ID --limit=50
gcloud monitoring policies list --project=PROJECT_ID
```

Troubleshooting Matrix

Symptom	Likely Cause	Evidence and Corrective Action
End-to-end latency rises but each service looks healthy	Queue backlog, workflow wait, retries, DNS/network, or hidden dependency	Use trace/business correlation across API, services, Pub/Sub/Eventarc, tasks, workflow, and data store.
Regional failover serves traffic but orders stall	Messaging, keys, secrets, data, scheduler, or workflow dependency not failed over	Validate dependency inventory and recovery sequence; reconcile in-flight business state.
Security policy blocks emergency release	No tested break-glass/exception workflow	Use approved time-bound exception with owner, evidence, compensating controls, and post-incident removal.
Cost exceeds forecast under load	Autoscaling bounds, logging, egress, cluster idle, or API/event fan-out	Use billing labels/export and workload metrics; tune bounds, retention, placement, and service selection.

Hands-on Lab 40: Final Google Cloud Cloud-Native Platform Capstone

Prerequisites

- An isolated project or project set, budget alerts, version control, synthetic data, and access to the services selected by the architecture.

Implementation

- Write business requirements, SLOs, RTO/RPO, classification, compliance, traffic, and budget assumptions.
- Create ADRs for runtime, API management, event routing, orchestration, and regional strategy.
- Build a trusted image pipeline and deploy at least one GKE service and one Cloud Run service by digest.
- Publish a private backend through API Gateway or document an Apigee implementation plan.
- Implement an idempotent Pub/Sub/Eventarc path with dead-letter handling and replay.
- Implement a Workflows process with Cloud Tasks and scheduled reconciliation.
- Add least-privilege workload identities, secrets, private networking, dashboards, alerts, and runbooks.
- Run load, canary rollback, duplicate delivery, dependency failure, and one regional or simulated control-plane game day.
- Produce a cost report, risk register, evidence index, and executive architecture briefing.

Validation Evidence

- Every component maps to a requirement and owner.
- Artifacts, identities, API calls, events, and workflow executions are traceable.
- Failure tests meet or clearly quantify SLO/RTO/RPO gaps.
- Cleanup, rollback, and incident runbooks are executable.

Cleanup

- Destroy all chargeable training resources after exporting evidence; verify with Asset Inventory and billing data.

Interview Questions and Model Answers

1. How do you choose between GKE and Cloud Run?

Compare required control, Kubernetes ecosystem, workload shape, portability, team capability, scaling, and operational burden.

2. What is the most important event-driven design property?

Idempotent processing with observable retry, dead-letter, and reconciliation behavior.

3. How do you prove a platform is production ready?

Through traceable requirements, controlled delivery, least privilege, load/failure tests, recovery evidence, SLOs, runbooks, and ownership.

4. What makes an architecture enterprise-grade?

Governed boundaries and repeatable operations that preserve security, reliability, cost, and change control across teams.

Chapter Summary

- A cloud-native platform is an integrated operating system for delivery, not a list of products.
- Runtime and integration choices require explicit trade-offs.
- Qualification must include failure, security, recovery, and cost evidence.
- Portfolio evidence should demonstrate engineering judgment and operational maturity.

Part D Enterprise Project Portfolio

Project	Required Outcome
Trusted multi-runtime release platform	Build once, sign/attest, promote by digest, enforce policy across GKE and Cloud Run, and demonstrate rollback.
GKE platform baseline	Regional cluster, workload identity, Gateway API, Network Policy, autoscaling, policy, backup, and upgrade game day.
Cloud Run SaaS cell	Private service, canary revisions, Direct VPC egress, bounded scaling, regional failover design, and unit-cost dashboard.
API product platform	Versioned contract, API Gateway implementation, Apigee decision model, quotas, analytics, consumer onboarding, and deprecation plan.
Event-driven order system	Pub/Sub, Eventarc, idempotency, dead-letter, replay, schemas/contracts, and end-to-end tracing.
Durable workflow platform	Workflows, Cloud Tasks, callbacks/compensation, Scheduler reconciliation, and failure-injection tests.

Project	Required Outcome
Container security incident	Compromised builder identity, affected digest discovery, policy block, rollback, rebuild, and evidence preservation.
Multi-cluster GKE design	Fleet registration, Config Sync, policy, service discovery, multi-cluster gateway, DR, and cost comparison.
Serverless dependency protection	Cloud Run/functions autoscaling limits, queue flow control, database connection budget, and overload game day.
Cloud-native FinOps review	Artifact storage, cluster utilization, serverless instance/concurrency, egress, logs, API and event unit economics.

Senior Interview Drill Bank

Scenario	Strong Answer Direction
A signed image is vulnerable. Can it deploy?	Signing proves identity/integrity, not safety. Apply policy based on vulnerability severity, exploitability, exceptions, and compensating controls.
GKE control plane is healthy but users see 503.	Trace load balancer, Gateway/Ingress, Service endpoints, readiness, application, and dependencies; control-plane health is insufficient.
Cloud Run scales rapidly and database fails.	Bound maximum instances, concurrency, and pool size; add queue/backpressure and size dependency capacity for failure states.
A function charges a customer twice.	Assume redelivery; use business idempotency key and atomic state before the external side effect.
API Gateway authenticates users. Why does backend need auth?	Defense in depth and direct-path protection; gateway identity and user/resource authorization remain distinct.
Pub/Sub backlog count is stable but oldest age rises.	New and old work may be mixed or one key is stuck; inspect age distribution, ordering keys, retries, and dependency latency.
Eventarc trigger is active with no delivery.	Check source event existence, filters, region, provider/audit configuration, service-agent IAM, and destination invoker role.
Workflow retry duplicates an order.	Retry only idempotent steps; use operation keys, transactional state, and compensation/reconciliation.
A scheduled job ran twice.	Scheduler/retry/manual overlap is possible; lock on business period and make the operation idempotent.
When should a team avoid Kubernetes?	When the workload fits a managed runtime and Kubernetes control does not justify cluster/platform complexity.

Command Reference

Area	Examples
Artifact Registry	gcloud artifacts repositories list; gcloud artifacts docker images list IMAGE_PATH --include-tags
GKE	gcloud container clusters list; gcloud container clusters describe NAME --region=REGION
Kubernetes	kubectl get events -A; kubectl auth can-i VERB RESOURCE --as=IDENTITY
Cloud Run	gcloud run services list; gcloud run revisions list --service=NAME --region=REGION
Cloud Run jobs	gcloud run jobs list --region=REGION; gcloud run jobs executions list --job=NAME --region=REGION
Cloud Run functions	gcloud run functions list --region=REGION; gcloud functions list --regions=REGION
API Gateway	gcloud api-gateway gateways list --location=REGION; gcloud api-gateway api-configs list --api=API
Pub/Sub	gcloud pubsub topics list; gcloud pubsub subscriptions describe NAME
Eventarc	gcloud eventarc triggers list --location=REGION; gcloud eventarc triggers describe NAME --location=REGION
Workflows	gcloud workflows list --location=REGION; gcloud workflows executions list NAME --location=REGION
Cloud Tasks	gcloud tasks queues list --location=REGION; gcloud tasks list --queue=QUEUE --location=REGION
Cloud Scheduler	gcloud scheduler jobs list --location=REGION; gcloud scheduler jobs run NAME --location=REGION
Asset and cleanup	gcloud asset search-all-resources --scope=projects/PROJECT_ID

Command groups, product names, and flags evolve. Confirm the installed gcloud components and current official command reference before production use. Always verify project, region, identity, digest, target, and deletion impact.

Production Readiness Checklist

Supply Chain

- ☐ Images are pinned by digest with source, builder, scan, SBOM, provenance, and approval evidence.
- ☐ Build, promotion, deploy, and runtime identities are separate.
- ☐ Deployment policy and emergency exception paths are tested.

Runtime

- ☐ GKE versus Cloud Run/function/job/worker choice is documented.
- ☐ Requests, limits, concurrency, autoscaling, quotas, disruption, and failure-state capacity are tested.
- ☐ Upgrade, canary, rollback, and deprecation ownership are explicit.

Identity and Network

- ☐ Workloads use short-lived identity without downloaded keys.
- ☐ Ingress, invocation, east-west, VPC egress, DNS, secrets, and firewall controls are least-privileged.
- ☐ Human admin and break-glass access are audited and time-bound.

API and Integration

- ☐ Contracts, versions, authentication, quotas, errors, and consumer ownership are governed.
- ☐ Events carry stable IDs, validated metadata, classification, and compatibility rules.
- ☐ Retries, idempotency, dead letters, replay, compensation, and reconciliation are tested.

Observability and Reliability

- ☐ SLOs cover user and asynchronous completion paths.
- ☐ Logs, metrics, traces, event IDs, task IDs, workflow executions, and business state correlate.
- ☐ Load, failure, restore/failover, and regional dependency game days have evidence.

Cost and Governance

- ☐ Labels, budgets, quotas, lifecycle cleanup, cluster/serverless utilization, egress, and log retention are reviewed.
- ☐ Platform templates and policies have owners, exception expiry, and compliance evidence.
- ☐ All lab and temporary resources have verified cleanup.

Glossary

Term	Definition
Admission control	Validation or mutation of Kubernetes resources before they are persisted.
Artifact digest	A content-addressed immutable identifier for an image or package.
Attestation	A signed statement about an artifact, build, test, or approval.
Autopilot	A GKE operating mode in which Google manages node infrastructure and many security/scaling settings.
Binary Authorization	A deploy-time policy service that evaluates image attestations and rules.
CloudEvents	An open specification for standardized event metadata.
Compensation	A business action that reverses or mitigates a previously completed distributed step.
Concurrency	The number of requests or units of work processed by one instance at the same time.
Dead-letter topic	A Pub/Sub topic used to isolate messages that cannot be processed after configured attempts.
Eventarc	Google Cloud managed event routing offered in Standard and Advanced editions.
Fleet	A logical grouping of GKE clusters for consistent management capabilities.

Term	Definition
Idempotency	The property that repeating an operation does not create additional unintended effects.
Ordering key	A Pub/Sub key that requests ordered delivery for related messages within documented scope.
PodDisruptionBudget	A Kubernetes policy limiting voluntary disruption of available pods.
Provenance	Verifiable metadata describing how and from what inputs an artifact was built.
Revision	An immutable Cloud Run service configuration and image version.
SBOM	Software bill of materials listing components included in an artifact.
Workload Identity Federation for GKE	Short-lived identity-based access from Kubernetes workloads to Google Cloud APIs without static keys.
Workflow connector	A built-in Workflows integration for calling a Google Cloud API.
Worker pool	A Cloud Run resource for continuous background work without a load-balanced URL.

Official Documentation Index

Service	Official Documentation
Artifact Registry	https://docs.cloud.google.com/artifact-registry/docs
Container Analysis	https://docs.cloud.google.com/artifact-analysis/docs
Binary Authorization	https://docs.cloud.google.com/binary-authorization/docs
Google Kubernetes Engine	https://docs.cloud.google.com/kubernetes-engine/docs
GKE Autopilot	https://docs.cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview
Workload Identity Federation for GKE	https://docs.cloud.google.com/kubernetes-engine/docs/how-to/workload-identity
Cloud Run	https://docs.cloud.google.com/run/docs
Cloud Run functions	https://docs.cloud.google.com/functions/docs
API Gateway	https://docs.cloud.google.com/api-gateway/docs
Apigee	https://docs.cloud.google.com/apigee/docs
Pub/Sub	https://docs.cloud.google.com/pubsub/docs
Eventarc	https://docs.cloud.google.com/eventarc/docs
Workflows	https://docs.cloud.google.com/workflows/docs
Cloud Tasks	https://docs.cloud.google.com/tasks/docs

Service	Official Documentation
Cloud Scheduler	https://docs.cloud.google.com/scheduler/docs
Cloud Architecture Center - Applications	https://docs.cloud.google.com/architecture/applications

Google Cloud services change continuously. Recheck editions, regional availability, limits, pricing, deprecations, execution semantics, and command syntax against official documentation before production implementation.

END OF VOLUME 8D | NEXT: VOLUME 8E - GOOGLE CLOUD DEVOPS, OBSERVABILITY, SECURITY, GOVERNANCE, AND FINOPS